Studio Nx

JS 258 — 칸 안 조작 연결

2026-10-18

JavaScript 258 — 칸 안 조작 연결 (`wireControls` · `deviceOfCard` · `paintKnob`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 칸 안의 노브 · 페이더 · 팬을 값과 이어 줍니다 (FR-2187 · FR-245)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	257 조작 연결 · 259 플러그인 고르기 창

1. 하는 일

`wireControls` 는 펼침 칸 하나를 훑어 조작 자리마다 `bindControl` 을 답니다. 트랙 볼륨과 팬, 카드 노브, 겹 볼륨과 팬 네 가지입니다. 각 자리마다 값이 어디서 오고(트랙 값인지 장치 값인지) 저장할 때 어느 함수를 부를지 정해 줍니다. `deviceOfCard` 는 카드에 적힌 줄 번호로 담아 둔 장치를 찾고, `paintKnob` 은 노브의 각도와 값 글자를 그립니다.

2. 언제 쓰나

쓰는 곳	
`renderCells` (251)	칸을 그린 뒤 마지막에 부릅니다
`bindModalParam` (261)	모달 노브는 `paintKnob` 을 그대로 씁니다
`toggleBypass` · `toggleSolo` (262) · `wireCells` (263)	카드의 장치를 찾을 때 `deviceOfCard` 를 씁니다

3. 코드

```
export function wireControls(scope, track) {
  Array.prototype.forEach.call(scope.querySelectorAll('[data-live="track-volume"]'), function (el) {
    const box = el.closest('.sq-strip') || el.closest('.sq-cell');
    bindControl(el, {
      item: { min: -60, max: 6, step: 0.1 },
      value: Number(track.volume_db) || 0,
      pixels: 120,
      render: function (v) {
        const bar = el.querySelector('i');
        if (bar) bar.style.setProperty('--y', faderY(v) + '%');
        const text = box && box.querySelector('.sq-db');
        if (text) text.textContent = fmtDb(v);
      },
      commit: function (v) {
        track.volume_db = v;
        updateTrack(track, { volume_db: v }).catch(warn);
      },
    });
  });
```

```
Array.prototype.forEach.call(scope.querySelectorAll('[data-live="track-pan"]'), function (el) {
  bindControl(el, {
    item: { min: -1, max: 1, step: 0.01 },
    value: Number(track.pan) || 0,
    pixels: 100,
    render: function (v) { el.style.setProperty('--p', panDeg(v) + 'deg'); },
    commit: function (v) {
      track.pan = v;
      updateTrack(track, { pan: v }).catch(warn);
    },
  });
});

Array.prototype.forEach.call(scope.querySelectorAll('.sq-fx[data-device-kind]'), function (card) {
  const device = deviceOfCard(track, card);
  const plugin = device ? pluginById(device.plugin_id) : null;
  if (!device || !plugin) return;
  const values = withDefaults(plugin, device.params);

  Array.prototype.forEach.call(card.querySelectorAll('.sq-knob[data-param-key]'), function (el) {
    const item = schemaItem(plugin, el.dataset.paramKey + '');
    if (!item) return;
    bindControl(el, {
      item: item,
      value: values[item.key],
      pixels: 130,
      render: function (v) { paintKnob(el, item, v); },
      commit: function (v) {
        const next = withDefaults(plugin, device.params);
        next[item.key] = v;
        device.params = next;
        updateDevice(track, card.dataset.deviceKind, device, { params: next }).catch(warn);
      },
    });
  });

  Array.prototype.forEach.call(card.querySelectorAll('[data-live="layer-volume"]'), function (el) {
    bindControl(el, {
      item: { min: -60, max: 6, step: 0.1 },
      value: Number(device.volume_db) || 0,
      pixels: 100,
      render: function (v) {
        const bar = el.querySelector('i');
        if (bar) bar.style.setProperty('--v', volPct(v) + '%');
        const out = el.querySelector('.sq-irow__db');
        if (out) out.textContent = fmtDb(v);
      },
      commit: function (v) {
        device.volume_db = v;
        updateDevice(track, card.dataset.deviceKind, device, { volume_db: v }).catch(warn);
      },
    });
  });

  Array.prototype.forEach.call(card.querySelectorAll('[data-live="layer-pan"]'), function (el) {
    bindControl(el, {
```

```
        item: { min: -1, max: 1, step: 0.01 },
        value: Number(device.pan) || 0,
        pixels: 80,
        render: function (v) { el.style.setProperty('--p', panDeg(v) + 'deg'); },
        commit: function (v) {
          device.pan = v;
          updateDevice(track, card.dataset.deviceKind, device, { pan: v }).catch(warn);
        },
      });
    });
  });
}

/** 카드의 장치 찾기 (devices 캐시에서) */
function deviceOfCard(track, card) {
  const d = devices[track.track_id];
  if (!d) return null;
  if (card.dataset.trackPluginId) {
    const id = Number(card.dataset.trackPluginId);
    return d.effects.filter(function (x) { return x.track_plugin_id === id; })[0] || null;
  }
  const id = Number(card.dataset.trackInstrumentId);
  return d.instruments.filter(function (x) { return x.track_instrument_id === id; })[0] || null;
}

/** 노브 그리기 — 각도와 값 글자 */
export function paintKnob(el, item, v) {
  const ring = el.querySelector('i');
  if (ring) ring.style.setProperty('--a', knobAngle(item, v) + 'deg');
  const out = el.querySelector('.sq-knob__val');
  if (out) out.textContent = fmtValue(item, v);
}
```

문법 풀이

식	뜻
`scope.querySelectorAll('[data-live="track-volume"]')`	칸 안의 볼륨 자리를 모두 찾습니다
`el.closest('.sq-strip') || el.closest('.sq-cell')`	dB 글을 어디서 찾을지 정합니다(믹서 줄인지 칸인지)
`item: { min: -60, max: 6, step: 0.1 }`	트랙 값은 스키마가 없어 범위를 직접 적습니다
`scope.querySelectorAll('.sq-fx[data-device-kind]')`	카드마다 한 번씩 돕니다
`el.dataset.paramKey + ''`	노브에 적힌 파라미터 키를 글자로 꺼냅니다
`withDefaults(plugin, device.params)`	저장 값에 기본값을 깔아 완전한 값으로 만듭니다
`next[item.key] = v`	사본에 새 값만 넣고 통째로 보냅니다
`deviceOfCard`	카드의 줄 번호로 장치를 찾습니다

4. 왜 이렇게

선택	이유
값 → 저장 → 캐시 갱신	담아 둔 장치를 고쳐 두어야 다음 그리기에서 값이 맞습니다

선택	이유
칸마다 다시 묶음	값이 바뀌면 칸을 다시 그리므로 묶음도 새로 필요합니다
`devBound` 로 한 번만	다시 그려도 같은 요소에 두 번 묶이지 않게 `bindControl` 이 막습니다
카드 사본을 통째로 저장	서버가 params 를 통째로 받으므로 빠진 키 없이 보내야 합니다

5. 시험

Bass Synth 카드 노브를 끌어 값이 바뀌고, 놓은 뒤 `GET /tracks/4639/plugins` 의 해당 줄 `params` 가 바뀐 것을 확인했습니다. 아래 믹서 줄 페이더와 펼침 칸 페이더가 같은 트랙 값을 함께 보고 있는 것도 확인했습니다.